

UNIVERSITATEA NAȚIONALĂ DE ȘTIINȚĂ ȘI TEHNOLOGIE
POLITEHNICA BUCUREȘTI

FACULTATEA DE ELECTRONICĂ, TELECOMUNICAȚII ȘI TEHNOLOGIA
INFORMAȚIEI

PROGRAMUL DE MASTERAT: INGINERIA INFORMAȚIEI ȘI A
SISTEMELOR DE CALCUL

PROIECT

Platformă educațională SQL cu Prisma

Modelare de date, ORM și integrare full-stack

Coordonator științific:

S.I. Dr. Ing. Valentin Pupezescu

Student:

Ing. Cristi Miloiu

411-IISC

BUCUREȘTI

2026

CUPRINS

I	Introducere și viziune generală	1
II	Fundamentare teoretică și analiză tehnologică	2
III	Specificații și fluxuri funcționale	4
IV	Arhitectura aplicației	6
V	Modelarea datelor și integrarea Prisma	7
VI	Documentarea API-ului backend	19
VII	Securitate, performanță și riscuri	20
VIII	Concluzii	22
	Bibliografie	22

I. INTRODUCERE ȘI VIZIUNE GENERALĂ

Digitalizarea procesului educațional a condus la apariția unui număr semnificativ de platforme de învățare. Totuși, în zona SQL, multe soluții rămân fie prea teoretice, fie prea puțin personalizabile. Proiectul `sql-learning-platform` pornește de la ideea că învățarea SQL este eficientă doar atunci când utilizatorul execută interogări reale pe un context de date controlat, iar progresul este urmărit coerent.

Tema proiectului este Prisma, aleasă nu doar ca utilitar tehnic, ci ca pilon de arhitectură. Prisma oferă o punte stabilă între modelul relațional (PostgreSQL) și codul TypeScript al backend-ului. Prin abordarea *schema-first + generate client*, aplicația obține consistență de tipuri, elimină o clasă largă de erori runtime și simplifică mentenanța pe termen lung.

A. Obiectivele documentației

Documentația are patru obiective principale:

- 1) descrierea completă a arhitecturii software și a fluxurilor funcționale;
- 2) explicarea utilizării Prisma ca ORM și strat de persistență;
- 3) justificarea alegerilor de design privind securitatea și scalabilitatea;
- 4) oferirea unei baze academice solide pentru evaluare și extindere ulterioară.

B. Contextul tehnologic

Aplicația este organizată în model client-server. Frontend-ul este implementat cu React + Vite, iar backend-ul cu Bun + Elysia. Persistența este asigurată de PostgreSQL și Prisma. Autentificarea se bazează pe Google OAuth, iar sesiunile sunt reprezentate prin JWT semnate server-side.

C. Viziune generală asupra proiectului

Scopul aplicației: Platforma are rolul de a oferi un mediu unificat pentru:

- parcurgerea lecțiilor SQL în ordine pedagogică;
- execuția de comenzi SQL pe o bază de date individuală;
- urmărirea progresului per lecție și per utilizator;
- separarea strictă a datelor personale față de datele didactice.

Componente principale: La nivel structural, proiectul conține:

- **client:** interfață React pentru autentificare, listă de lecții, lecții individuale și setări;
- **server:** API REST construit cu Elysia, responsabil de autentificare, logică de business și integrarea cu Prisma;
- **prisma schema + migrations:** contractul de date și istoricul schimbărilor de structură;
- **docker-compose:** orchestrare pentru rulare locală sau evaluare.

Rolul central al Prisma: Prisma joacă un rol central în:

- a) definirea modelelor entitate-relație într-un limbaj declarat clar;
- b) generarea unui client TypeScript cu API predictibil;
- c) aplicarea migrațiilor în medii de dezvoltare și producție;
- d) implementarea operațiilor `find`, `create`, `update`, `upsert`, `deleteMany` în mod sigur.

II. FUNDAMENTARE TEORETICĂ ȘI ANALIZĂ TEHNOLOGICĂ

A. Fundamentare teoretică: Prisma ca ORM modern

Ce este un ORM: Un ORM (*Object-Relational Mapping*) permite maparea entităților obiectuale din cod pe tabele relaționale. Beneficiile standard includ reducerea codului boilerplate, consistență semantică și abstractizarea dialectelor SQL. Costurile tipice sunt curbă de învățare, risc de query-uri ineficiente în lipsa unei modelări corecte și uneori pierdere de control fin la nivel SQL brut.

Particularități Prisma: Prisma adresează limitări istorice ale ORM-urilor tradiționale prin:

- **schema explicită:** toate modelele sunt declarate în `schema.prisma`;
- **type safety:** clientul generat folosește tipuri stricte în TypeScript;
- **migrații versionate:** schimbările bazei sunt reproducibile;
- **DX ridicat:** syntaxă clară, onboarding rapid, integrare bună cu editorul.

Prisma ca „DB layer”: În contextul proiectului curent, Prisma este „stratul canonic” pentru baza de date aplicației: autentificare utilizator, progres, lecții, metadata ale bazei personale. Chiar dacă utilizatorul poate executa SQL direct pe propria bază, datele nucleu ale platformei rămân guvernate de Prisma.

B. Comparație Prisma cu alternative ORM

Criterii de evaluare: Comparația dintre Prisma și alte opțiuni uzuale (de exemplu TypeORM, Sequelize, Knex + layer manual) poate fi formulată pe următoarele criterii:

- claritatea modelării datelor;
- siguranța la compilare și ergonomia API-ului;
- maturitatea migrațiilor;
- controlul asupra interogărilor complexe;
- costul de onboarding pentru echipe mici.

Observații comparative: Prisma se diferențiază prin schema declarativă ușor de citit și prin clientul generat care reflectă fidel modelele. TypeORM oferă o abordare *decorator-based* direct în clasele TypeScript, dar poate deveni dificil de menținut când numărul de entități crește, iar convențiile nu sunt standardizate intern. Sequelize este matur, dar orientat istoric pe pattern-uri mai puțin stricte tipic pentru TypeScript modern.

Knex, ca query builder, oferă control foarte bun la nivel SQL, însă necesită un efort suplimentar pentru modelare, validare și mapare de tipuri. În contextul unui proiect educațional full-stack,

unde viteza de iterație este critică, Prisma oferă echilibrul cel mai bun între productivitate și rigoare.

Justificarea alegerii finale: Alegerea Prisma este justificată de:

- 1) integrarea naturală cu TypeScript;
- 2) reducerea numărului de defecte legate de schema DB;
- 3) migrații transparente și ușor de versionat;
- 4) claritate pentru evaluare academică și colaborare.

C. Analiză critică a alegerii Prisma

Avantaje observate: Integrarea Prisma în acest proiect aduce:

- viteză mare de implementare a fluxurilor CRUD;
- reducerea bug-urilor de tip mapare câmpuri;
- evoluție predictibilă a schemei;
- cod mai accesibil pentru echipe mixte (backend + full-stack).

Limitări și compromisuri: Prisma nu elimină nevoia de înțelegere SQL. Pentru optimizare avansată este necesară analiză directă a planurilor de execuție. În plus, endpoint-urile ce rulează SQL brut rămân în afara acoperirii ORM, necesitând guvernanta suplimentară.

Decizie tehnică argumentată: Pentru scopul educațional al proiectului, Prisma reprezintă un echilibru optim între productivitate, claritate și control. Costurile de adoptare sunt mici, iar beneficiile de consistență sunt semnificative.

III. SPECIFICAȚII ȘI FLUXURI FUNCȚIONALE

A. Fluxuri funcționale esențiale

Autentificare Google + persistență utilizator: Endpoint-ul `/api/auth/google` validează credentialul și rulează:

- `upsert` în tabela `User` pe baza `googleId`;
- generare JWT cu `sub=user.id`.

Prisma simplifică semnificativ acest flux: o singură operație acoperă prima autentificare și logările ulterioare.

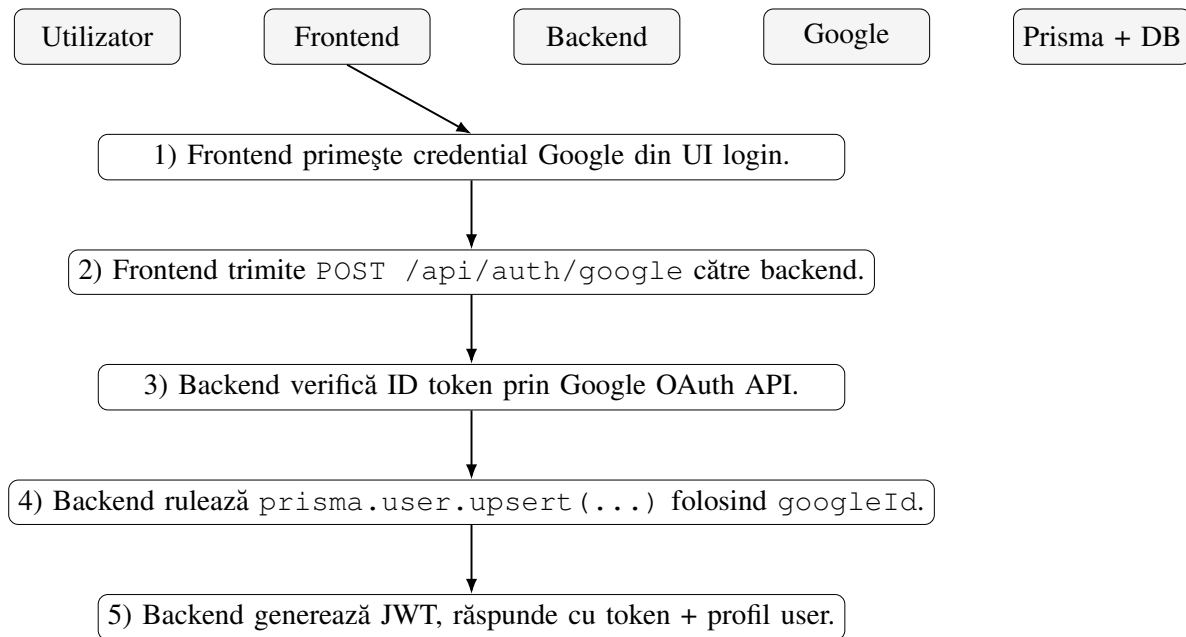


Figura 1. Secvență de autentificare Google și sincronizare utilizator cu Prisma.

Catalog lecții + sincronizare persistentă: La accesarea endpoint-urilor de lecții, backend-ul realizează `seedCatalog()`:

- `upsert` pentru fiecare lecție din catalogul static;
- `deleteMany` pentru lecții eliminate din sursa de adevăr.

Acest mecanism menține consistența între definiția didactică și baza persistentă.

Progres utilizator: La finalizarea unei lecții se execută `upsert` în `LessonProgress`. Modelul permite reprezentarea separata a:

- setup-ului inițial (`setupAt`);
- completării didactice (`completed`, `completedAt`).

Gestionarea bazei de date personale: Conexiunea utilizatorului este:

- 1) primita prin API;
- 2) criptată cu AES-GCM pe server;
- 3) salvată în `UserDatabase.encryptedUrl`;
- 4) decriptată doar la nevoie pentru operații live.

B. Specificații funcționale detaliate

Use case-uri principale: Din perspectivă didactică, sistemul poate fi descris prin următoarele use case-uri:

- 1) **Autentificare:** utilizatorul inițiază login prin Google și primește o sesiune JWT.
- 2) **Vizualizare lecții:** utilizatorul vede lista ordonată a lecțiilor cu starea progresului.
- 3) **Studiu lecție:** utilizatorul accesează conținutul unei lecții și eventual setup-ul SQL asociat.
- 4) **Execuție SQL:** utilizatorul rulează comenzi SQL pe baza sa personală.
- 5) **Marcare progres:** utilizatorul marchează lecția ca finalizată.
- 6) **Administrare conexiune DB:** utilizatorul salvează sau generează o conexiune.

Actorii sistemului: Actorii sunt:

- **Utilizator final:** consumatorul platformei educaționale;
- **Server API:** orchestrator fluxuri, autentificare, validare și persistență;
- **PostgreSQL aplicație:** stochează datele interne ale platformei;
- **PostgreSQL utilizator:** mediu practic de rulare SQL;
- **Google Identity:** furnizor extern de identitate.

Cerințe funcționale sintetice:

ID	Cerință
F1	Sistemul trebuie să permită autentificarea pe bază de cont Google și să creeze automat profilul intern la prima logare.
F2	Sistemul trebuie să ofere listă de lecții cu ordonare deterministică și statut de progres per utilizator.
F3	Sistemul trebuie să permită marcarea finalizării lecțiilor cu persistarea timestamp-ului.
F4	Sistemul trebuie să permită configurarea și stocarea securizată a conexiunii DB a utilizatorului.
F5	Sistemul trebuie să permită execuția SQL live pe baza utilizatorului și afișarea rezultatului.
F6	Sistemul trebuie să permită inițializarea dataset-ului de lecție prin scripturi setup idempotente.

IV. ARHITECTURA APLICAȚIEI

A. Model arhitectural

Arhitectura urmează modelul *Frontend SPA + Backend API + DB*. Clientul comunică exclusiv cu API-ul serverului. Serverul:

- validează token-urile;
- aplică reguli de autorizare;
- citește/scrie date prin Prisma;
- gestionează conexiunile dinamice spre baza fiecărui utilizator pentru execuție SQL.

B. Avantajul separării responsabilităților

Separarea între *data platform* (Prisma + DB intern) și *learning sandbox* (DB utilizator) previne contaminarea datelor, simplifică auditul de securitate și permite extinderea independentă a fiecărei componente.

C. Flux operațional de nivel înalt

Fluxul tipic este:

- 1) utilizatorul se autentifică prin Google OAuth;
- 2) serverul verifică ID token-ul și face `upsert` în tabela `User`;
- 3) frontend-ul primește JWT și apelează endpoint-uri protejate;
- 4) lecțiile sunt listate și asociate cu progresul;
- 5) utilizatorul configurează conexiunea PostgreSQL personală;
- 6) execuția SQL are loc controlat, iar progresul este persistat.

V. MODELAREA DATELOR ȘI INTEGRAREA PRISMA

A. Modelarea datelor cu Prisma

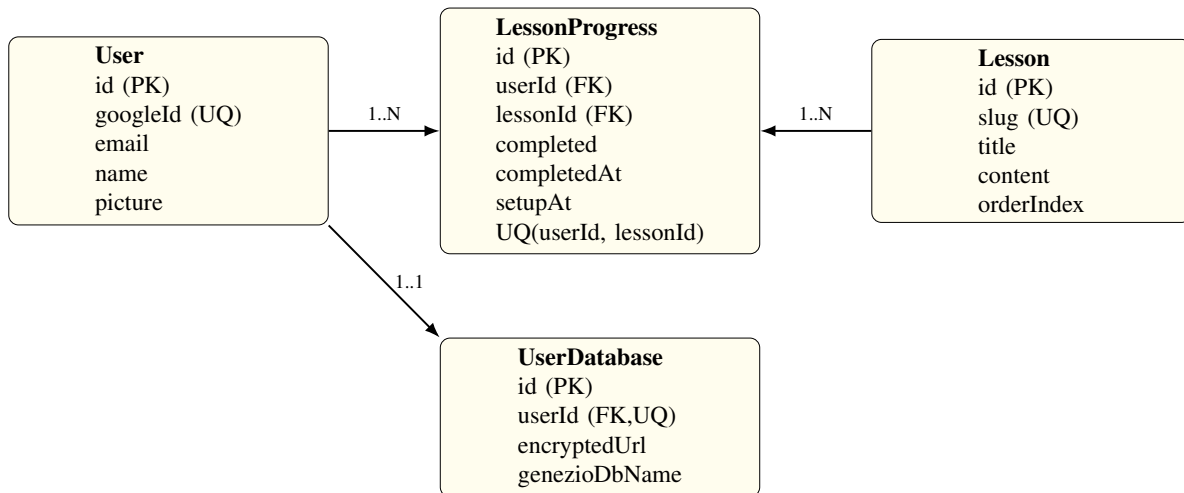


Figura 2. Modelul relațional derivat din `schema.prisma`.

Configurare inițială: Fișierul `schema.prisma` conține generatorul Prisma Client și datasource-ul PostgreSQL:

Listing 1. Schema completă a bazei de date (`schema.prisma`)

```

generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

model User {
  id          String    @id @default(cuid())
  googleId   String    @unique
  email      String
  name       String?
  picture    String?
  createdAt  DateTime @default(now())
  updatedAt  DateTime @updatedAt

  lessonsProgress LessonProgress[]

```

```

    userDatabase      UserDatabase?
}

model Lesson {
  id          String    @id @default(cuid())
  slug        String    @unique
  title       String
  description  String?
  content      String    // markdown or HTML
  orderIndex  Int        @default(0)
  createdAt   DateTime  @default(now())
  updatedAt   DateTime  @updatedAt

  progress LessonProgress[]
}

model LessonProgress {
  id          String    @id @default(cuid())
  userId      String
  lessonId    String
  completed   Boolean    @default(false)
  completedAt DateTime?
  setupAt     DateTime?
  createdAt   DateTime  @default(now())
  updatedAt   DateTime  @updatedAt

  user User @relation(fields: [userId], references: [id], onDelete:
    Cascade)
  lesson Lesson @relation(fields: [lessonId], references: [id], onDelete:
    Cascade)

  @@unique([userId, lessonId])
  @@index([userId])
  @@index([lessonId])
}

model UserDatabase {
  id          String    @id @default(cuid())
  userId      String    @unique
  encryptedUrl String?   // null when DB was created via Genezio but URL
                        not returned
  genezioDbName String?  // e.g. learn-8261d355fb36; one DB per user
  createdAt   DateTime  @default(now())

```

```

    updatedAt      DateTime @updatedAt

    user User @relation(fields: [userId], references: [id], onDelete:
        Cascade)

    @@index([userId])
}

```

Această structură are două implicații importante:

- mediile diferite (dev/prod) sunt controlate din variabile de mediu, nu hardcodate;
- clientul ORM este regenerat sincronizat cu schema.

Modelul User: Entitatea `User` reține identitatea Google și informații de profil:

- `googleId` este unic și sursa primară de reconciliere la login;
- `id` este cheie internă (`cuid()`) folosită pentru relații;
- relațiile cu progresul și baza de date a utilizatorului sunt explicite.

Modelul Lesson: Modelul `Lesson` exprimă unitatea didactică. Câmpurile `slug` și `orderIndex` sunt esențiale: primul asigură identificare stabilă în URL, al doilea garantează ordonarea pedagogică independent de ID.

Modelul LessonProgress: Acest model captează starea utilizatorului per lecție. Constrângerea compusă `@@unique([userId, lessonId])` impune unicitatea unei singure înregistrări de progres pentru fiecare pereche utilizator-lecție. Indexurile separate pe `userId` și `lessonId` optimizează interogări frecvente.

Modelul UserDatabase: Modelul este proiectat pentru a stoca **doar versiunea criptată** a URL-ului de conexiune (`encryptedUrl`), plus un identificator opțional pentru baza generată automat. Acest design reduce expunerea accidentală a credentialelor.

Integritatea relațională: Toate relațiile cheie folosesc `onDelete: Cascade`. Astfel, eliminarea unui utilizator propagă automat spre progres și metadata asociate, evitând date orfane și simplificând mentenanța.

B. Prisma Client în backend

Instanțiere sigură: Backend-ul folosește un singleton Prisma Client cu protecție pentru medii de dezvoltare cu hot reload:

Listing 2. Singleton Prisma Client

```
const globalForPrisma = globalThis as unknown as { prisma: PrismaClient };

export const prisma =
  globalForPrisma.prisma ??
  new PrismaClient({
    log: process.env.NODE_ENV === "development"
      ? ["query", "error", "warn"]
      : ["error"],
  });

if (process.env.NODE_ENV !== "production") globalForPrisma.prisma = prisma;
```

Prin acest pattern se evită multiplicarea conexiunilor în sesiuni repetitive de dezvoltare.

Pattern-uri de acces folosite: Proiectul utilizează un set coerent de operații:

- upsert pentru idempotentă (User, Lesson, LessonProgress, UserDatabase);
- findUnique pentru acces determinist;
- findMany cu select pentru payload minim;
- deleteMany pentru sincronizare catalog lecții.

De ce upsert este critic: Upsert reduce condițiile de cursă și elimină bifurcațiile „dacă există, altfel creează”. În fluxul login Google, de exemplu, permite actualizarea profilului fără blocuri separate de cod și menține consistența identității.

C. Migrații și management de schemă

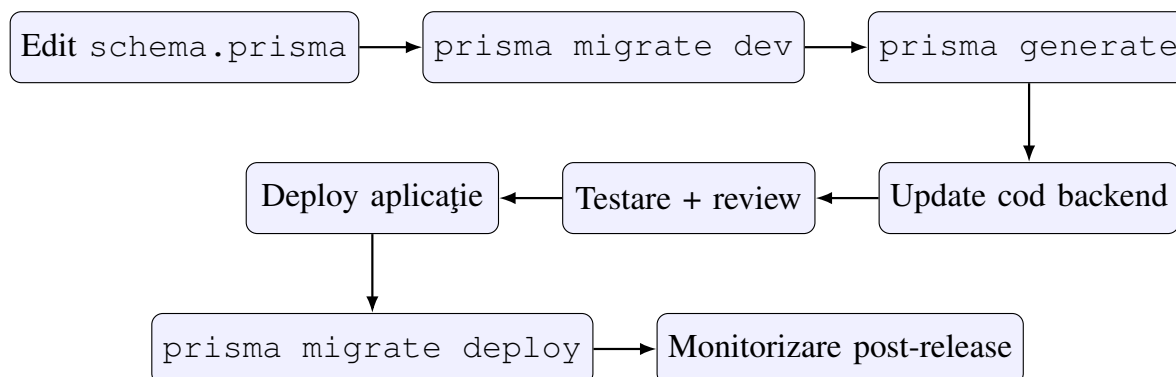


Figura 3. Flux recomandat pentru lifecycle-ul Prisma în proiect.

Comenzi de lucru: Backend-ul expune scripturi explicite:

- `bun run db:generate` – generează Prisma Client;
- `bun run db:migrate:dev` – creează/aplică migrații în dezvoltare;
- `bun run db:migrate` – aplică migrații existente în deployment;
- `bun run db:studio` – interfață inspectare date.

Strategie recomandată: Fluxul recomandat pentru o schimbare de model:

- 1) actualizare `schema.prisma`;
- 2) rulare migrare în dev;
- 3) validare cu date reale;
- 4) commit migrare + schema + cod dependent;
- 5) deploy și `migrate deploy`.

Riscuri și mitigare: Principalele riscuri la migrare sunt: incompatibilități de tip, lock pe tabele mari și regresii de performanță. Prisma reduce aceste riscuri prin istoricul explicit al migrațiilor, dar responsabilitatea finală rămâne la echipa de dezvoltare (review, backup, testare).

Sincronizare ORM vs. Migrații generate: Prisma oferă opțiunea de sincronizare automată (prin `prisma db push`), o funcționalitate care aliniază direct baza de date cu schema fără să stocheze istoric. Cu toate acestea, în proiectul curent, această abordare este strict dezactivată pentru ciclurile formale (fiind tolerată doar temporar la prototipare iterativă). Nu este considerată o practică bună executarea automată a primei migrări în producție, deoarece se pierde vizibilitatea asupra tipurilor de date efective, asupra indecșilor săpați și a clauzelor de constraint generate de baza de date.

În schimb, aplicația folosește sistemul strict de migrații (`prisma migrate dev`), care stochează snapshot-uri exacte de cod SQL. De exemplu, migrarea inițială (fișierul `20250307000000_init`) cuprinde un cod generat manual/validat de developer:

Listing 3. Extras din migrarea inițială (Crearea entităților principale)

```

-- CreateTable
CREATE TABLE "User" (
  "id" TEXT NOT NULL,
  "googleId" TEXT NOT NULL,
  "email" TEXT NOT NULL,
  "name" TEXT,
  "picture" TEXT,
  "createdAt" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  "updatedAt" TIMESTAMP(3) NOT NULL,

  CONSTRAINT "User_pkey" PRIMARY KEY ("id")
);

-- CreateTable
CREATE TABLE "Lesson" (
  "id" TEXT NOT NULL,
  "slug" TEXT NOT NULL,
  "title" TEXT NOT NULL,
  "description" TEXT,
  "content" TEXT NOT NULL,
  "orderIndex" INTEGER NOT NULL DEFAULT 0,
  "createdAt" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  "updatedAt" TIMESTAMP(3) NOT NULL,

  CONSTRAINT "Lesson_pkey" PRIMARY KEY ("id")
);

```

Un exemplu puternic al modului în care sistemul de migrații reflectă evoluția de business a aplicației este modificarea logică de "one database per user". În acest punct, coloana `encryptedUrl` a devenit opțională (NULL) până când infrastructura din spate furnizează URL-ul, adăugându-se și coloana de nume intern `genezioDbName`. Această alterare este salvată ireversibil în `20250307120000_one_db_per_user`:

Listing 4. Evoluția `UserDatabase` prin migrare incrementală

```

-- One DB per user: allow row without URL (Genezio created but URL not in
  response);
-- store genezio name to avoid creating a second DB
ALTER TABLE "UserDatabase" ADD COLUMN IF NOT EXISTS "genezioDbName" TEXT;
ALTER TABLE "UserDatabase" ALTER COLUMN "encryptedUrl" DROP NOT NULL;

```

D. Data dictionary extins

Tabela User:

Câmp	Tip	Semnificație
id	String (cuid)	Identificator intern platformă, cheie primară.
googleId	String	Identificator unic furnizat de Google Identity.
email	String	Email utilizator, folosit și în token payload.
name	String?	Nume afișat în UI (opțional).
picture	String?	URL avatar Google (opțional).
createdAt	DateTime	Moment creare cont intern.
updatedAt	DateTime	Ultima actualizare automata.

Tabela Lesson:

Câmp	Tip	Semnificație
id	String (cuid)	Cheie primară internă.
slug	String	Identificator semantic, unic în API și rutare UI.
title	String	Titlu lecție.
description	String?	Rezumat scurt pentru lista lecțiilor.
content	String	Conținut principal (markdown/HTML).
orderIndex	Int	Poziție în secvența didactică.
createdAt,	DateTime	Audit temporal minimal.
updatedAt		

Tabela LessonProgress:

Câmp	Tip	Semnificație
id	String (cuid)	Cheie primară.
userId	String	FK spre User.
lessonId	String	FK spre Lesson.
completed	Boolean	Flag finalizare lecție.
completedAt	DateTime?	Timestamp finalizare.
setupAt	DateTime?	Timestamp setup dataset lecție.

Tabela UserDatabase:

Câmp	Tip	Semnificație
id	String (cuid)	Cheie primară.
userId	String	FK unic spre User.
encryptedUrl	String?	URL conexiune criptat AES-GCM.
genezioDbName	String?	Nume DB creată din API extern.
createdAt, updatedAt	DateTime	Metadata temporale.

E. Prisma în profunzime: internals, decizii și pattern-uri

Feature-uri Prisma folosite efectiv în proiect: Pentru a ancora documentația în implementarea reală din `sql-learning-platform`, mai jos sunt funcționalitățile Prisma utilizate direct în codul backend:

Feature Prisma folosit	Cum este folosit în proiect
<code>schema.prisma</code> (generator datasource)	Definește contractul de date și conectarea la PostgreSQL prin <code>DATABASE_URL</code> .
<code>model + relații</code> (@relation)	Modelele <code>User</code> , <code>Lesson</code> , <code>LessonProgress</code> , <code>UserDatabase</code> descriu domeniul aplicației și legăturile dintre entități.
Constrângeri (@unique, @@unique)	<code>googleId</code> este unic, iar progresul este garantat unic pe perechea (<code>userId</code> , <code>lessonId</code>).
Indexuri (@@index)	Accelerează interogările frecvente pe <code>lessonProgress</code> și <code>userDatabase</code> .
<code>onDelete: Cascade</code>	Asigură curățarea automată a datelor dependente la ștergeri de entități părinte.
<code>findUnique</code>	Folosit pentru acces determinist la utilizator, lecții, progres, configurație DB utilizator.
<code>findMany</code>	Folosit pentru listarea lecțiilor și progresului, cu selectarea controlată a câmpurilor.
<code>upsert</code>	Folosit intensiv pentru login idempotent, seed lecții, progres lecții, salvare conexiune DB utilizator.
<code>deleteMany</code>	Sincronizează catalogul persistent cu sursa de adevăr (<code>LESSON_CATALOG</code>).
<code>select</code>	Reduce payload-ul trimis către client (în special în listări).

Prisma Client singleton	Evită proliferarea conexiunilor în development (<i>hot reload safe pattern</i>).
Migrații Prisma	Gestionate cu <code>prisma migrate dev</code> și <code>prisma migrate deploy</code> .

Feature-uri „related” (nefolosite direct, dar recomandate): Următoarele capabilități nu sunt nucleare în implementarea curentă, dar sunt foarte relevante pentru versiuni viitoare:

- `$transaction` pentru fluxuri multi-mutare strict atomice;
- Prisma middleware/extensii pentru audit și politici centralizate;
- query profiling mai avansat și instrumentare runtime;
- read replicas și optimizări pentru trafic ridicat.

Interpretare practică: ce este must-have vs nice-to-have: Pentru acest proiect, **must-have** sunt deja implementate: modelare, relații, constrângeri, `upsert`, `find*`, migrații și singleton client. Elementele **nice-to-have** sunt cele „related”, utile mai ales la extindere pe volum mai mare sau cerințe operaționale stricte.

Modelul conceptual Prisma: Prisma poate fi privit ca un lanț „schema → client generat → query SQL executat”. Pentru un proiect ca `sql-learning-platform`, acest lanț aduce un avantaj major: fiecare schimbare din modelul de date propagă explicit în cod, iar compilatorul TypeScript semnalează devreme zonele afectate.

La nivel practic, modelul conceptual se traduce astfel:

- 1) **Definire structură** în `schema.prisma`;
- 2) **Migrare** pentru a sincroniza PostgreSQL cu noua versiune;
- 3) **Generare client** pentru API-ul type-safe;
- 4) **Consum controlat** în rute backend.

Anatomia unei interogări Prisma: O interogare Prisma trece prin mai multe etape:

- validare shape input în runtime + tipuri la compilare;
- transformare în reprezentare internă de query;
- generare SQL specific dialectului PostgreSQL;
- mapare rezultat SQL în obiecte JS/TS.

Această conductă explică de ce erori comune (câmp greșit, `include/select invalid`) sunt prinse mai devreme decât într-un strat SQL manual.

De ce schema actuală este bine adaptată proiectului: Schema existentă este minimală, dar bine normalizată pentru domeniul educațional:

- `User` gestionează identitatea;
- `Lesson` stochează obiectul educațional;
- `LessonProgress` modelează relația many-to-many cu stare;
- `UserDatabase` separă metadatele conexiunii personale.

Din punct de vedere al proiectării, cea mai importantă decizie este `@@unique([userId, lessonId])`. Aceasta encodează direct regula de business „un singur progres per utilizator per lecție”, prevenind inconsistențe chiar și sub concurență.

Pattern-ul upsert ca mecanism de consistență: Proiectul utilizează `upsert` în mai multe fluxuri. Tehnic, `upsert` are 3 avantaje:

- a) reduce numărul de round-trip-uri DB;
- b) evită ramuri `if (exists) ... else ...` vulnerabile la race condition;
- c) concentrează regula de sincronizare într-un singur punct de cod.

Exemplu relevant din proiect:

Listing 5. Upsert robust pentru progres

```
await prisma.lessonProgress.upsert({
  where: { userId_lessonId: { userId, lessonId: lesson.id } },
  create: { userId, lessonId: lesson.id, completed: true, completedAt: new
    Date() },
  update: { completed: true, completedAt: new Date() },
});
```

Acest pattern este preferabil în endpoint-uri cu trafic concurent (de exemplu click-uri repetate pe „Complete lesson”).

Select vs Include: control pe payload și cost: `select` și `include` sunt instrumente esențiale de optimizare:

- `select` limitează coloanele returnate și reduce payload-ul;
- `include` aduce relații, dar poate crește semnificativ costul interogării.

În proiect, folosirea `select` pentru listing-ul de lecții este decizia corectă, deoarece UI-ul nu are nevoie de content complet la listare. O extensie recomandată este standardizarea acestui principiu la toate endpoint-urile de citire.

Constrângeri Prisma și reguli de business: Prisma permite exprimarea explicită a constrângerilor:

- unicitate simplă (`@unique`);
- unicitate compusă (`@@unique`);
- indexare (`@@index`);
- integritate referențială (`@relation + onDelete`).

O bună practică este ca orice regulă de business stabilă să existe și la nivel DB (nu doar în cod). Schema proiectului respectă deja acest principiu pentru progres.

Migrații: strategie tehnică matură: Dincolo de comenzile standard, partea importantă este strategia:

- 1) **migrații mici, frecvente**, nu schimbări monolitice;
- 2) **evitarea rename-urilor brute** pe tabele mari în ore de vârf;
- 3) **compatibilitate backward temporară** în deploy gradual;
- 4) **testare migrație pe copie de date realiste**.

Pentru acest proiect, când se vor introduce modele noi (Quiz, Submission), recomandarea este deploy în doi pași: mai întâi schemă + cod compatibil vechi/nou, apoi curățare câmpuri depășite.

Tranzacții Prisma: când trebuie folosite: Deși proiectul folosește operații atomice punctuale, o zonă naturală de extindere este folosirea `$transaction` pentru operații multi-entitate. Regula simplă: dacă două sau mai multe mutații trebuie să reușească „împreună sau deloc”, atunci tranzacția este obligatorie.

Exemplu propus:

Listing 6. Pattern tranzacțional recomandat

```
await prisma.$transaction(async (tx) => {
  const lesson = await tx.lesson.findUnique({ where: { slug } });
  if (!lesson) throw new Error("Lesson not found");

  await tx.lessonProgress.upsert({
    where: { userId_lessonId: { userId, lessonId: lesson.id } },
    create: { userId, lessonId: lesson.id, setupAt: new Date() },
    update: { setupAt: new Date() },
  });

  await tx.user.update({
    where: { id: userId },
    data: { updatedAt: new Date() },
  });
});
```

Concurență, idempotență și retry: În sisteme web reale, aceleași request-uri pot fi trimise de mai multe ori (retries client, timeout-uri, dublu click). Prisma ajută, dar designul endpoint-ului rămâne esențial:

- folosiți `upsert` unde operația este natural idempotentă;
- tratați explicit erorile de unicitate (de tip P2002) și transformați-le în răspunsuri HTTP coerente;
- evitați pattern-ul `findFirst -> create` fără protecție tranzacțională.

Prisma și observabilitate: Observabilitatea bună înseamnă să poți răspunde la „de ce este lent endpoint-ul X”. Pentru asta:

- 1) activați loguri Prisma diferențiat dev/prod;
- 2) adăugați corelație request-id în loguri API;
- 3) măsurați separat timpul de query DB vs logică aplicație;
- 4) analizați periodic query-urile cele mai lente.

Conexiuni și pooling: Chiar dacă proiectul folosește singleton Prisma Client, la scalare reală problema principală devine managementul conexiunilor:

- prea multe instanțe server pot satura PostgreSQL;
- funcțiile serverless pot crea burst-uri de conexiuni;
- lipsa unui pooler (ex. PgBouncer) crește latența sub trafic.

Recomandarea pentru versiune de producție extinsă: utilizare pooling dedicat, limitare concurență pe endpoint-uri costisitoare, backpressure și timeouts stricte.

Boundary-ul dintre Prisma și SQL brut: Proiectul are intenționat endpoint-uri cu SQL brut pentru scop educațional. Tehnic, este importantă delimitarea:

- **Prisma** pentru datele sistemului (User, Lesson, Progress);
- **SQL brut controlat** pentru exerciții pe DB utilizator.

Această separare este corectă. În schimb, pentru a reduce riscuri, se recomandă: whitelist de comenzi SELECT/WITH în mod „safe”, timeout query la nivel driver, limită de rânduri returnate.

Anti-pattern-uri de evitat în evoluția proiectului:

- 1) adăugarea de câmpuri „generic JSON” fără motiv solid;
- 2) folosirea excesivă a `include` pe relații adânci;
- 3) endpoint-uri care returnează direct obiecte DB complete;
- 4) migrații mari și riscante, lansate rar;
- 5) lipsa testelor de regresie la schimbări de schemă.

Checklist tehnic Prisma un release:

Verificare	Criteriu de acceptare
Schema validă	<code>schema.prisma</code> compilează și clientul se generează fără warning-uri critice.
Migrare sigură	Migrația rulează pe staging cu date realiste și fără pierdere de integritate.
Contract API stabil	Endpoint-urile nu expun câmpuri sensibile noi accidental.
Performanță	Query-urile nou introduse au latență acceptabilă sub sarcină de test.
Observabilitate	Există loguri suficiente pentru diagnosticul incidentelor post-release.

Concluzie tehnică pe Prisma: Pentru `sql-learning-platform`, Prisma nu este doar un ORM convenabil, ci mecanismul care formalizează datele sistemului, reduce complexitatea de dezvoltare și oferă bază pentru evoluție sigură. Pe măsură ce proiectul crește, maturitatea va veni din disciplină de migrare, observabilitate și utilizarea atentă a tranzacțiilor.

VI. DOCUMENTAREA API-ULUI BACKEND

A. Endpoint-uri de autentificare

Endpoint	Metodă	Descriere
<code>/api/auth/google</code>	POST	Primește credential Google, verifică token, execută <code>upsert</code> în <code>User</code> , returnează JWT + user profile.
<code>/api/auth/me</code>	GET	Verifică JWT și returnează profilul curent din baza internă.

B. Endpoint-uri de lecții

Endpoint	Metodă	Descriere
<code>/api/lessons</code>	GET	Returnează lecțiile ordonate plus progresul utilizatorului.
<code>/api/lessons/:slug</code>	GET	Returnează detalii lecție + setup meta + progres.

<code>/api/lessons/:slug/setup</code>	POST	Rulează setup SQL de lecție pe DB utilizatorului (opțional forțat).
<code>/api/lessons/:slug/complete</code>	POST	Marchează lecția ca finalizată.

C. Endpoint-uri user database

Endpoint	Metodă	Descriere
<code>/api/user-db</code>	GET	Returnează starea configurării DB a utilizatorului.
<code>/api/user-db</code>	POST	Primește URL conexiune și îl salvează criptat.
<code>/api/user-db/generate</code>	POST	Creează DB nouă prin integrare externă și salvează metadata.
<code>/api/user-db/decrypted</code>	GET	Returnează URL decriptat (util intern controlat).
<code>/api/user-db/live</code>	GET	Listează tabele sau date sample dintr-un tabel specificat.
<code>/api/user-db/run</code>	POST	Rulează SQL brut și returnează rezultat tabular.

VII. SECURITATE, PERFORMANȚĂ ȘI RISCURI

A. Securitate și protecția datelor

Principii aplicate: Platforma aplică principii fundamentale:

- minimizarea datelor sensibile persistate;
- separarea autentificării de autorizare;
- validare input la endpoint-uri;
- control CORS prin origine configurabilă.

Prisma și securitatea: Prisma contribuie la securitate indirect prin:

- API type-safe, reducând erori de mapare;
- evitarea SQL dinamic pentru operații de business;
- control centralizat al relațiilor și constrângerilor.

Zone sensibile: Există două zone care necesită atenție ridicată:

- a) endpoint-ul de rulare SQL liber (`/api/user-db/run`);
- b) endpoint-ul de introspecție live (`/api/user-db/live`).

Fiind intenționat orientate spre învățare, aceste endpoint-uri trebuie acompaniate de politici de limitare a impactului (rate limiting, sandboxing, timeout-uri stricte).

B. Performanță și scalabilitate

Caracteristici curente: Din perspectiva Prisma, proiectul este eficient pentru volum mic-mediu:

- interogări simple cu indexuri relevante;
- payload-uri selective prin `select`;
- operații idempotente eficiente.

Potențiale optimizări: Pentru scalare:

- 1) paginare la endpoint-ul lecțiilor;
- 2) caching pentru catalog static;
- 3) monitorizare query plan PostgreSQL;
- 4) eventuală introducere de read replicas;
- 5) pooling robust în deployment distribuit.

Discuție despre costul ORM: La scară foarte mare, overhead-ul ORM poate deveni relevant. În acest proiect, compromisurile sunt favorabile: dezvoltare rapidă, claritate și siguranță. Pentru cazuri punctuale, se poate combina Prisma cu query SQL controlat.

C. Matrice de risc

Riscuri tehnice:

Risc	Probabilitate	Impact	Mitigare
Expunere SQL periculos prin endpoint run	Medie	Ridicat	Validare input, roluri, limitare statement-uri, timeout, audit.
Creștere latență la query-uri neindexate	Medie	Mediu	Revizie indexuri, EXPLAIN ANALYZE, monitorizare.
Regresii la schimbări de schemă	Medie	Ridicat	Migrații testate în staging, backup înainte de deploy.
Scurgere accidentală de secrete în loguri	Scăzută	Ridicat	Redactare loguri, politici de secret scanning.
Dependență de provider extern OAuth	Medie	Mediu	Retry, fallback UX, cache profil local minimal.

Riscuri organizaționale:

- lipsa disciplinei de migrare poate produce deriva schemelor;
- lipsa testelor automate reduce viteza sigură de evoluție;
- rotația secretelor fără plan poate invalida accesul la date.

VIII. CONCLUZII

Proiectul `sql-learning-platform` demonstrează că Prisma poate fi utilizat eficient în dublu rol: motor de modelare/migrare și ORM de producție pentru un backend TypeScript modern. Prin schema clară, operații idempotente, constrângeri relaționale bine definite și integrare directă în fluxul API, Prisma asigură o fundație robustă pentru o platformă educațională orientată spre date.

Din perspectiva calității software, proiectul atinge obiective importante: consistență a datelor, securitate a informațiilor sensibile, extensibilitate și mentenanță facilă. Există zone ce necesită maturizare (monitorizare, rate limiting, testare automata extinsă), însă fundația este solidă, iar alegerea Prisma este justificată tehnic și pedagogic.

Codul sursă integral al platformei, incluzând arhitectura frontend, backend și schema completă Prisma, este disponibil open-source pe GitHub la adresa: <https://github.com/cristim67/sql-learning-platform>.

BIBLIOGRAFIE

- [1] Prisma, “Prisma documentation,” <https://www.prisma.io/docs>, 2026, accesat: 2026-05-09.
- [2] PostgreSQL Global Development Group, “Postgresql documentation,” <https://www.postgresql.org/docs/>, 2026, accesat: 2026-05-09.
- [3] Google, “Openid connect and oauth 2.0,” <https://developers.google.com/identity>, 2026, accesat: 2026-05-09.
- [4] ElysiaJS, “Elysia framework documentation,” <https://elysiajs.com/>, 2026, accesat: 2026-05-09.
- [5] Bun, “Bun runtime and tooling,” <https://bun.sh/docs>, 2026, accesat: 2026-05-09.
- [6] IEEE, “Ieee editorial style manual,” in *IEEE Publishing Guidelines*, 2023.

ANEXA A: EXEMPLE PRISMA DIN PROIECT

A.1 Model simplificat

```
model LessonProgress {  
  id          String    @id @default(cuid())  
  userId      String  
  lessonId    String  
  completed   Boolean   @default(false)  
  completedAt DateTime?  
  setupAt     DateTime?  
  
  user User @relation(fields: [userId], references: [id], onDelete: Cascade)  
  lesson Lesson @relation(fields: [lessonId], references: [id], onDelete: Cascade)  
  
  @@unique([userId, lessonId])  
  @@index([userId])  
  @@index([lessonId])  
}
```

A.2 Upsert tipic

```
await prisma.lessonProgress.upsert({
  where: { userId_lessonId: { userId, lessonId: lesson.id } },
  create: { userId, lessonId: lesson.id, completed: true, completedAt: new
    Date() },
  update: { completed: true, completedAt: new Date() },
});
```

A.3 Verificarea token-ului Google OAuth

```
const response = await fetch(
  'https://oauth2.googleapis.com/tokeninfo?id_token=${idToken}'
);
const payload = await response.json();

if (!response.ok || payload.aud !== process.env.GOOGLE_CLIENT_ID) {
  throw new Error("Token invalid sau expirat.");
}

// Upsert bazat pe googleId
const user = await prisma.user.upsert({
  where: { googleId: payload.sub },
  update: { name: payload.name, picture: payload.picture },
  create: {
    googleId: payload.sub,
    email: payload.email,
    name: payload.name,
    picture: payload.picture,
  },
});
```

ANEXA B: OPERAȚIUNI DE PERSISTENȚĂ ȘI TRANZACȚII

B.1 Seeding idempotent (Catalog de lecții)

```
// Sincronizarea catalogului hardcodat cu baza de date Prisma
async function seedCatalog() {
  for (const l of LESSON_CATALOG) {
    await prisma.lesson.upsert({
      where: { slug: l.slug },
      create: { slug: l.slug, title: l.title, content: l.content,
        orderIndex: l.orderIndex },
      update: { title: l.title, content: l.content, orderIndex:
        l.orderIndex },
    });
  }
  // Curatare lectii obsolete pastrand integritatea referentiala
  const slugs = LESSON_CATALOG.map((l) => l.slug);
  await prisma.lesson.deleteMany({ where: { slug: { notIn: slugs } } });
}
```

B.2 Tranzacții native pe mediul utilizatorului (pg driver)

```
// Executarea de scripturi DDL pe DB-ul dedicat al utilizatorului folosind
pg client
const client = await getUserDbConnection(userId);
await client.connect();
try {
  await client.query("BEGIN"); // start tranzactie nativa
  for (const stmt of def.setupSql) {
    await client.query(stmt);
  }
  await client.query("COMMIT");
} catch (e) {
  await client.query("ROLLBACK");
  throw e;
} finally {
  await client.end();
}
```

B.2 Tranzacții interactive (\$transaction)

```
// Crearea userului si a bazei de date asociate intr-un pas atomic
const result = await prisma.$transaction(async (tx) => {
  const newUser = await tx.user.create({
    data: {
      email: "student@example.com",
      googleId: "google-oauth-1234",
      name: "Student"
    }
  });

  const userDb = await tx.userDatabase.create({
    data: {
      userId: newUser.id,
      genezioDbName: `db_${newUser.id}`,
      encryptedUrl: "encrypted_string_aici..."
    }
  });

  return { user: newUser, db: userDb };
});
```